

Guía Didáctica: Promesas en JavaScript

Apartado 5: Conclusiones sobre Promesas y Programación Asíncrona

Explicación

Las **promesas en JavaScript** son fundamentales para la **programación asíncrona**, permitiendo:

- Ejecutar **tareas que tardan tiempo** sin bloquear la página.
- Manejar **resultados o errores** de forma clara con `then()`, `catch()` y `finally()`.
- Evitar el **callback hell** mediante **encadenamiento (chaining)**.
- Trabajar de forma natural con la **Fetch API** y otras operaciones asíncronas.

Ventajas de usar promesas:

1. Código más limpio y fácil de leer.
2. Separación clara entre **operaciones exitosas y errores**.
3. Posibilidad de encadenar operaciones dependientes.
4. Manejo paralelo de múltiples promesas con `Promise.all()`.

Ejemplo práctico 1: Resumen visual de una promesa completa

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Resumen Promesas</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">
</head>
<body class="container py-4">

<h1>Ejemplo 1: Resumen Promesa Completa</h1>
<button id="btnResumen1" class="btn btn-primary">Ejecutar
Promesa</button>
<div id="resultadoResumen1" class="mt-3"></div>

<script>
const btnResumen1 = document.getElementById('btnResumen1');
const resultadoResumen1 =
document.getElementById('resultadoResumen1');

btnResumen1.addEventListener('click', () => {
```

```

    resultadoResumen1.innerHTML = `<div class="alert alert-info">Estado: Pending...</div>`;

    new Promise((resolve, reject) => {
        const exito = Math.random() > 0.3;
        setTimeout(() => exito ? resolve("Promesa cumplida!") :
        reject("Promesa fallida!"), 2000);
    })
    .then(msg => resultadoResumen1.innerHTML = `<div class="alert alert-success">${msg}</div>`)
    .catch(err => resultadoResumen1.innerHTML = `<div class="alert alert-danger">${err}</div>`)
    .finally(() => console.log("Promesa finalizada"));
});
</script>

</body>
</html>

```

Explicación:

- Combina **pending**, **fulfilled/rejected** y **finally()** para mostrar un flujo completo.
- Ideal para que los alumnos vean el ciclo de vida completo de una promesa.

Ejemplo práctico 2: Encadenamiento + Fetch + Manejo de errores

```

<h2>Ejemplo 2: Fetch y Chaining</h2>
<button id="btnResumen2" class="btn btn-success">Cargar Usuario y Posts</button>
<div id="resultadoResumen2" class="mt-3"></div>

<script>
const btnResumen2 = document.getElementById('btnResumen2');
const resultadoResumen2 =
document.getElementById('resultadoResumen2');

btnResumen2.addEventListener('click', () => {
    fetch('https://jsonplaceholder.typicode.com/users/1')
        .then(response => {
            if(!response.ok) throw new Error("Error HTTP: " +
            response.status);
            return response.json();
        })
        .then(user => {
            resultadoResumen2.innerHTML = `<div class="alert alert-info">Usuario: ${user.name}</div>`;
            return
            fetch(`https://jsonplaceholder.typicode.com/posts?userId=${user.id}`);
        })
        .then(response => response.json())
        .then(posts => {
            let html = '<ul class="list-group">';
            posts.forEach(post => html += `<li class="list-group-item">${post.title}</li>`);
            html += '</ul>';
            resultadoResumen2.innerHTML += html;
        })
    });

```

```

    })
    .catch(err => resultadoResumen2.innerHTML = `<div class="alert
alert-danger">${err}</div>`)
    .finally(() => console.log("Operación finalizada"));
  });
</script>

```

Explicación:

- Combina encadenamiento y fetch para mostrar **uso real de promesas en peticiones asíncronas al servidor**.
- `catch()` captura cualquier error en la cadena.

Ejemplo práctico 3: Promise.all para operaciones paralelas

```

<h2>Ejemplo 3: Promise.all</h2>
<button id="btnResumen3" class="btn btn-info">Cargar Usuarios
Simultáneos</button>
<div id="resultadoResumen3" class="mt-3"></div>

<script>
const btnResumen3 = document.getElementById('btnResumen3');
const resultadoResumen3 =
document.getElementById('resultadoResumen3');

btnResumen3.addEventListener('click', () => {
  const urls = [
    'https://jsonplaceholder.typicode.com/users/1',
    'https://jsonplaceholder.typicode.com/users/2',
    'https://jsonplaceholder.typicode.com/users/3'
  ];

  Promise.all(urls.map(url => fetch(url).then(res => res.json())))
    .then(results => {
      let html = '';
      results.forEach(user => html += `<div class="alert alert-
info">Usuario: ${user.name}</div>`);
      resultadoResumen3.innerHTML = html;
    })
    .catch(err => resultadoResumen3.innerHTML = `<div class="alert
alert-danger">${err}</div>`);
  });
</script>

```

Explicación:

- `Promise.all()` permite ejecutar varias promesas simultáneamente.
- Muy útil cuando queremos **obtener varios datos al mismo tiempo** sin esperar secuencialmente.

Ejemplo práctico 4: Simulación completa de flujo asíncrono

```

<h2>Ejemplo 4: Flujo completo</h2>
<button id="btnResumen4" class="btn btn-secondary">Ejecutar
Flujo</button>
<div id="resultadoResumen4" class="mt-3"></div>

<script>
const btnResumen4 = document.getElementById('btnResumen4');
const resultadoResumen4 =
document.getElementById('resultadoResumen4');

btnResumen4.addEventListener('click', () => {
    resultadoResumen4.innerHTML = `<div class="alert alert-
info">Iniciando operación...</div>`;

    new Promise((resolve, reject) => setTimeout(() => resolve("Paso 1
completado"), 1000))
        .then(msg => {
            resultadoResumen4.innerHTML += `<div class="alert alert-
success">${msg}</div>`;
            return new Promise((resolve) => setTimeout(() =>
resolve("Paso 2 completado"), 1000));
        })
        .then(msg => {
            resultadoResumen4.innerHTML += `<div class="alert alert-
success">${msg}</div>`;
            return new Promise((resolve, reject) => setTimeout(() =>
reject("Paso 3 falló"), 1000));
        })
        .then(msg => resultadoResumen4.innerHTML += `<div class="alert
alert-success">${msg}</div>`)
        .catch(err => resultadoResumen4.innerHTML += `<div
class="alert alert-danger">${err}</div>`)
        .finally(() => resultadoResumen4.innerHTML += `<div
class="alert alert-secondary">Flujo finalizado</div>`);
    });
</script>

```

Explicación:

- Simula un **flujo completo de pasos asíncronos**, con éxito y fallo.
- Permite visualizar cómo se maneja la **secuencia y los errores**.

Ejercicios prácticos resueltos para alumnos

1. Crear un flujo de tres promesas encadenadas, mostrando resultados intermedios y finales.
 2. Usar `Promise.all()` para obtener datos de varios usuarios simultáneamente.
 3. Simular una serie de pasos de un proceso (por ejemplo, descarga → procesamiento → visualización) y mostrar estado en pantalla.
 4. Crear un fetch GET y POST encadenados, manejando errores con `catch()` y finalización con `finally()`.
-

Ejercicios propuestos para alumnos

1. Crear un flujo de promesas encadenadas que simulen el registro de un usuario, envío de email y confirmación final.
 2. Usar `fetch()` para obtener varios recursos al mismo tiempo y mostrarlos usando `Promise.all()`.
 3. Crear un flujo que combine operaciones exitosas y fallidas, capturando errores en `catch()`.
 4. Implementar un flujo completo que simule un carrito de compras: añadir productos → calcular total → enviar pedido, mostrando mensajes de estado en pantalla.
-

Conclusión general sobre Promesas y programación asíncrona

- Las promesas permiten **programación asíncrona limpia y estructurada**.
- Facilitan el manejo de **peticiones al servidor** y tareas que tardan tiempo.
- Combinadas con **fetch API**, son esenciales para **desarrollos web modernos**.
- Buenas prácticas: usar `catch()` para errores, `finally()` para limpieza, y `Promise.all()` para operaciones paralelas.
- Permiten **secuencias complejas y encadenadas** sin anidamiento profundo de callbacks.